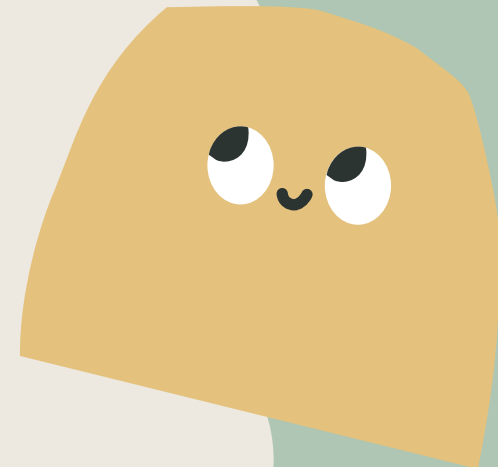
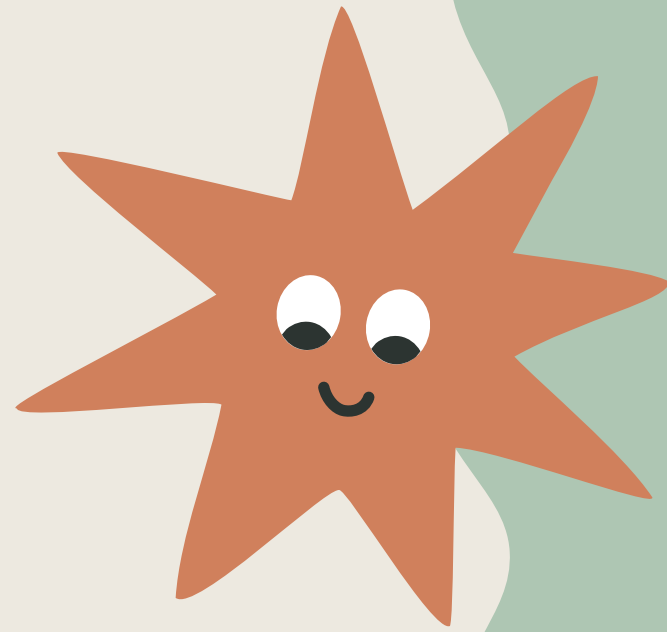


Singly Linked List: Introduction, Node Structure & Basic Operations

Lecture 8



Learning Outcomes (Lecture 8)

➤ At the end of this lecture, students will be able to:

- Explain the limitations of arrays.
- Explain why linked lists were introduced.
- Understand the structure of a node.
- Draw singly linked lists.
- Create nodes dynamically.
- Implement a linked list using classes.
- Perform traversal, insertion, deletion, searching, and updating.
- Compare arrays and linked lists.

Linked List

- A Linked List is a linear data structure in which elements (nodes) are stored in non-contiguous memory locations and connected using pointers.
- Important Features:
 - Linear
 - Dynamic
 - Non-contiguous memory
 - Pointer based

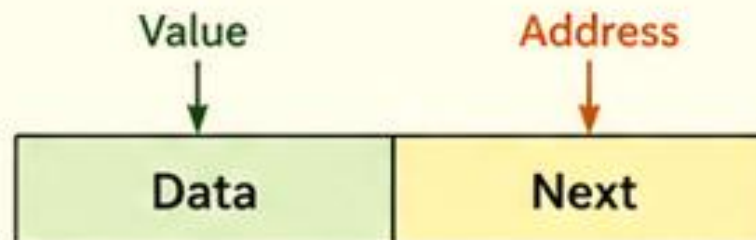
Node Structure

1. NODE STRUCTURE

A node is the basic building block of a linked list.
Each node contains two parts:

- Data: Stores the value.
- Next: Stores the address of the next node.

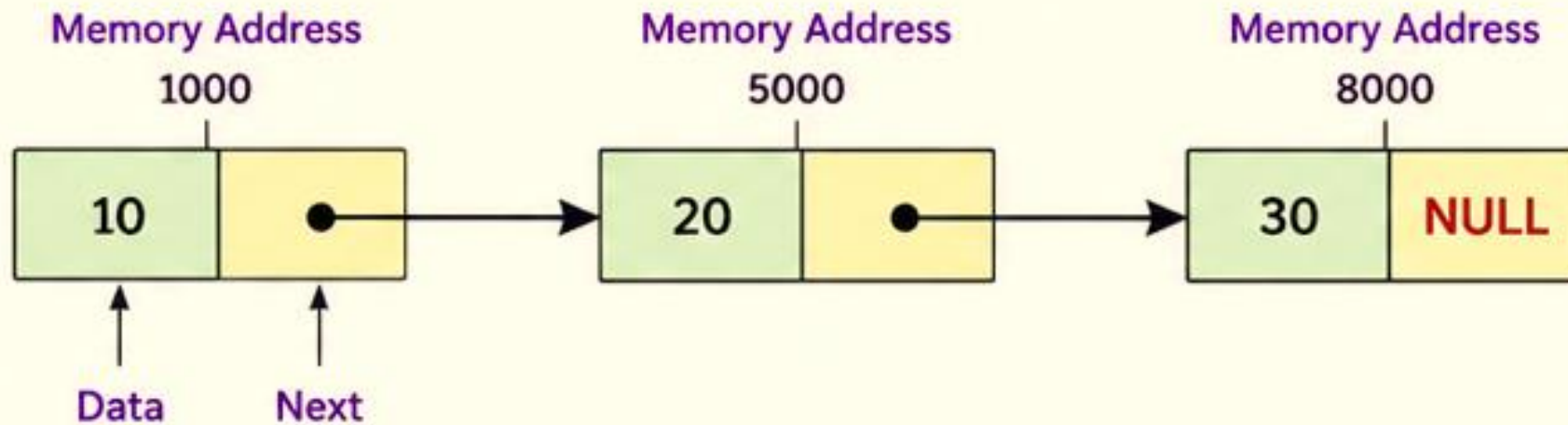
Visual Representation of a Node



Nodes in Memory

2. NODE IN MEMORY

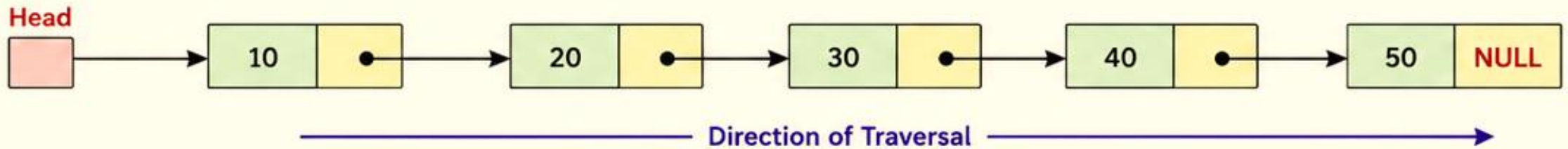
Nodes are stored in non-contiguous memory locations.



Linked List

3. LINKED LIST (EXAMPLE)

A linked list is a collection of nodes where each node points to the next node. The last node points to NULL, indicating the end of the list.



➤ Important Terms

- Node, Head, Next, NULL

Array vs Linked List

Array	Linked List
Contiguous memory	Non-contiguous memory
Fixed size	Dynamic size
Fast random access	Sequential access
Expensive insertion	Efficient insertion
Expensive deletion	Efficient deletion

➤ Which one is better?

Basic Operations on Linked List

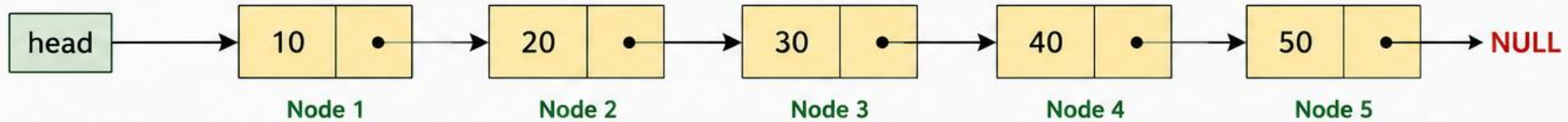
➤ Following are the basic operations supported by a Linked List:

- **Traverse** – Print all the elements of linked list one by one
- **Insertion (4 Cases)** – Adds before an element, adds after an element, adds at start, add at end of linked list
- **Deletion (3 Cases)** – Delete by value, delete from start, delete at the end
- **Search** – Searches a node by the value
- **Update** – Updates a node's value

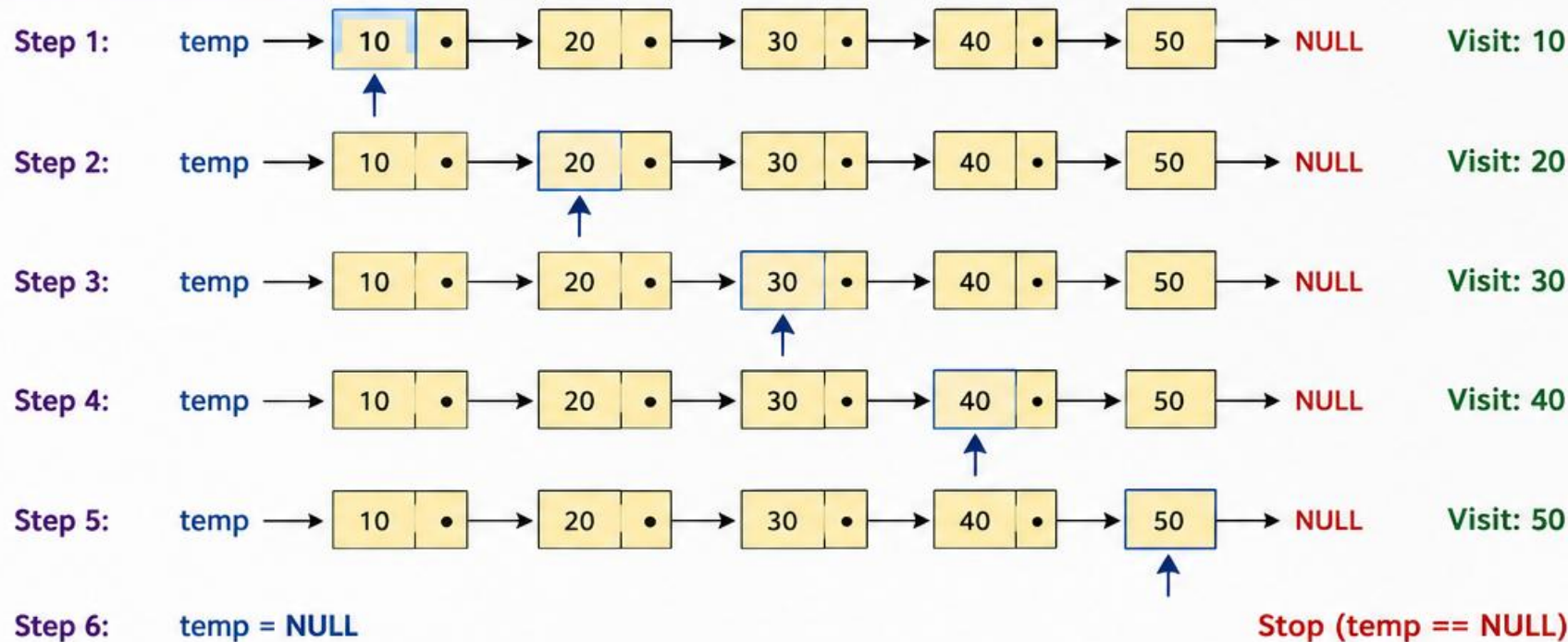
TRAVERSAL IN SINGLY LINKED LIST

Traversal means visiting each node of the linked list exactly once from the first node (head) to the last node (NULL).

LINKED LIST EXAMPLE



TRAVERSAL PROCESS



ALGORITHM

1. START
2. Create a pointer temp and assign head to it.
3. If temp is not NULL, then
 - a. Visit (process) the data part of temp.
 - b. Move temp to next node (temp = temp->next).
4. Repeat Step 3 until temp becomes NULL.
5. STOP

Traversal Time Complexity

TIME COMPLEXITY



If the linked list contains n nodes,
the time complexity of traversal is:

$$O(n)$$

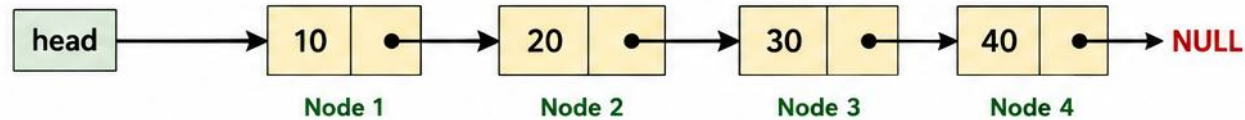
Why?

We must visit each node exactly once.
Therefore, the number of operations is directly
proportional to the number of nodes (n).

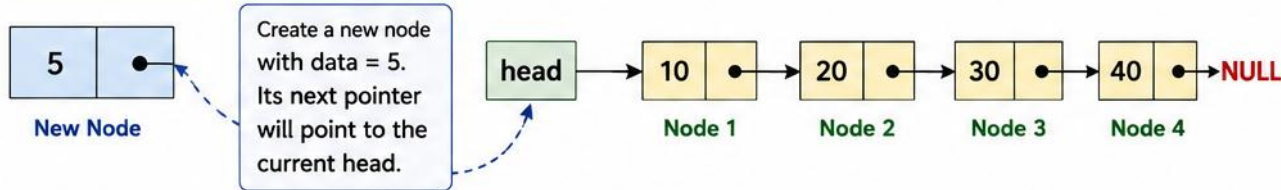
INSERT AT START (BEGINNING) IN SINGLY LINKED LIST

New node is inserted before the current head and then head is updated to point to the new node.

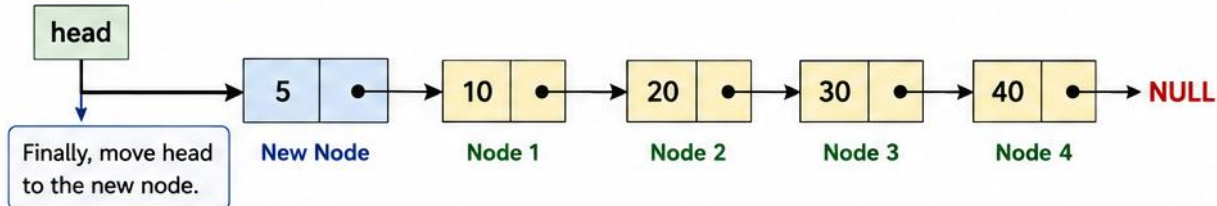
BEFORE INSERTION



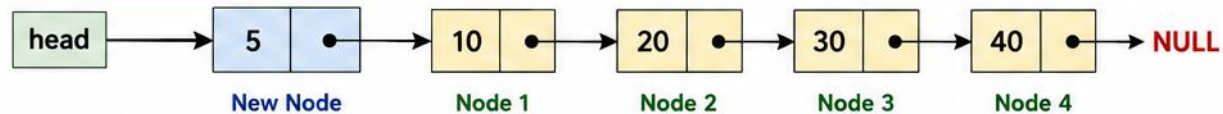
STEP 1: CREATE NEW NODE



STEP 2: UPDATE HEAD



AFTER INSERTION



New node is now the first node (head) of the linked list.

ALGORITHM

- 1 Create a new node.
- 2 Set the new node's next pointer to the current head.
- 3 Update head to point to the new node.
- 4 Insertion at start is complete.

TIME COMPLEXITY



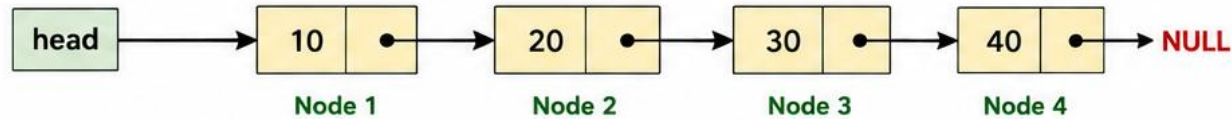
$O(1)$

Only a constant number of operations are performed, regardless of the number of nodes.

INSERT AT END IN SINGLY LINKED LIST

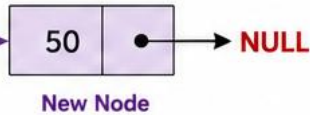
A new node is inserted after the last node. The last node's next pointer is updated to point to the new node, and the new node's next pointer is set to NULL.

BEFORE INSERTION

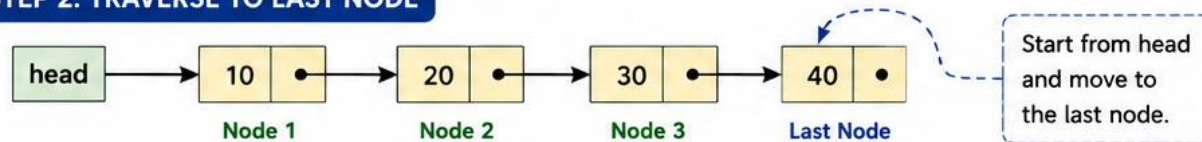


STEP 1: CREATE NEW NODE

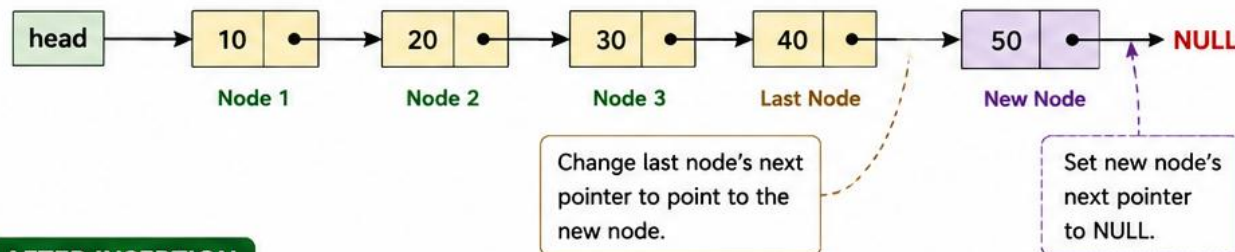
Create a new node with data = 50 and next = NULL



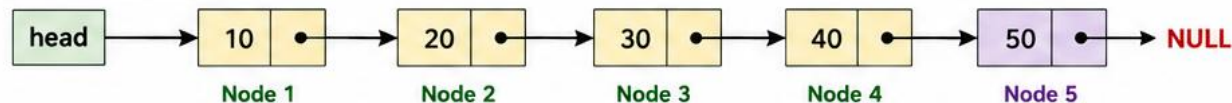
STEP 2: TRAVERSE TO LAST NODE



STEP 3: INSERT NEW NODE



AFTER INSERTION



ALGORITHM

- 1 Create a new node.
- 2 If the list is empty (head is NULL), make the new node as head.
- 3 Otherwise, traverse the list to reach the last node.
- 4 Set the last node's next pointer to the new node.
- 5 Set the new node's next pointer to NULL.
- 6 Insertion at end is complete.

TIME COMPLEXITY



$O(n)$

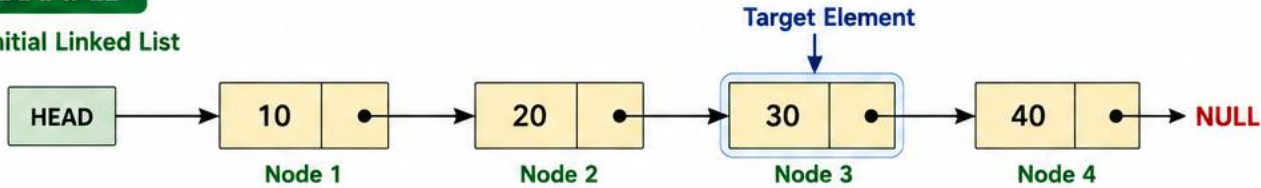
We may need to traverse all n nodes to reach the last node before insertion. (If we have a tail pointer, it becomes $O(1)$.)

INSERT BEFORE A GIVEN ELEMENT IN SINGLY LINKED LIST

A new node is inserted before the first occurrence of the given element.

EXAMPLE

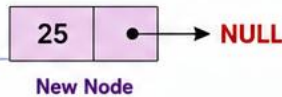
Initial Linked List



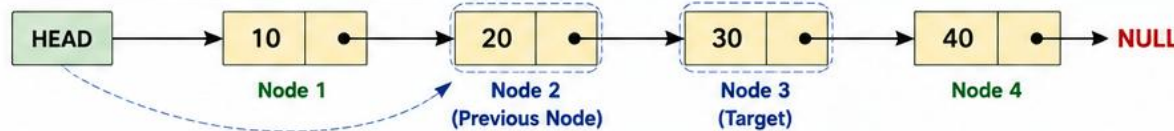
Operation:
Insert 25 before element 30 (first occurrence).

STEP 1: CREATE NEW NODE

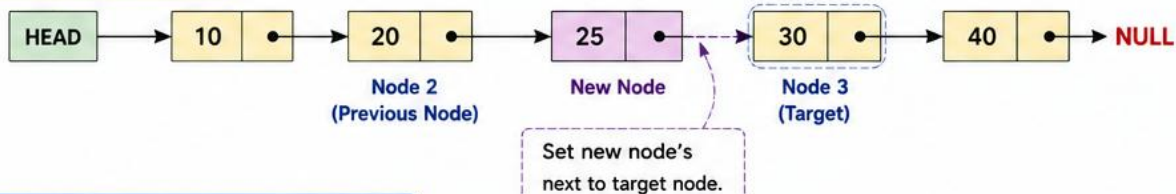
New node with
data = 25



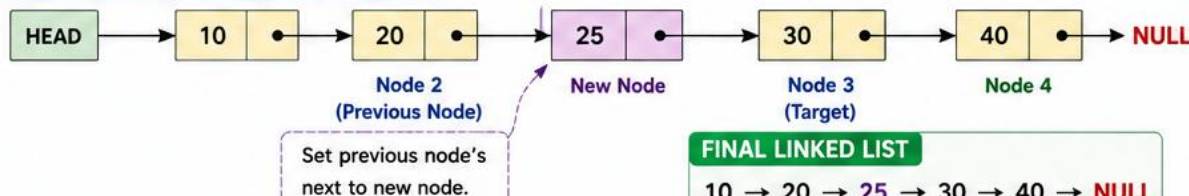
STEP 2: TRAVERSE TO FIND PREVIOUS NODE



STEP 3: LINK NEW NODE



STEP 4: UPDATE PREVIOUS NODE'S NEXT



ALGORITHM

- 1 Create a new node with the given data.
- 2 If the list is empty:
 - Set the new node as head and return.
- 3 If the given element is the head's data:
 - Set new node's next to head.
 - Update head to the new node and return.
- 4 Traverse the list to find the node whose next node contains the given element (previous node).
- 5 If the element is not found, do not insert the new node.
- 6 Set new node's next to previous node's next.
- 7 Set previous node's next to the new node.
- 8 Insertion before the given element is complete.

TIME COMPLEXITY



$O(n)$

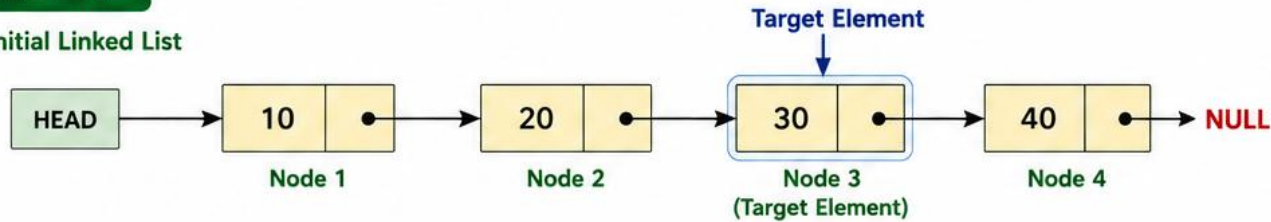
In the worst case, we may need to traverse all n nodes to find the given element or its previous node.

INSERT AFTER A GIVEN ELEMENT IN SINGLY LINKED LIST

A new node is inserted after the first occurrence of the given element.

EXAMPLE

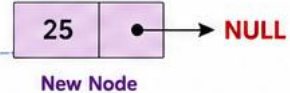
Initial Linked List



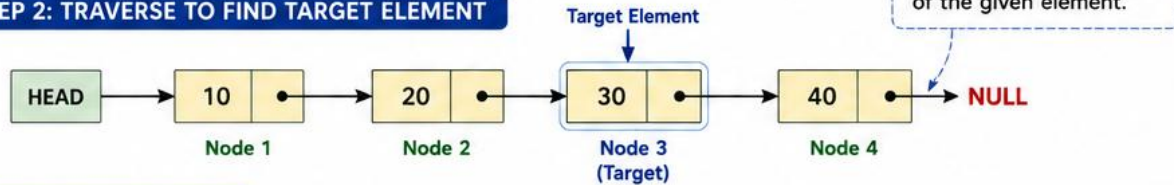
Operation:
Insert 25 after element 30 (first occurrence).

STEP 1: CREATE NEW NODE

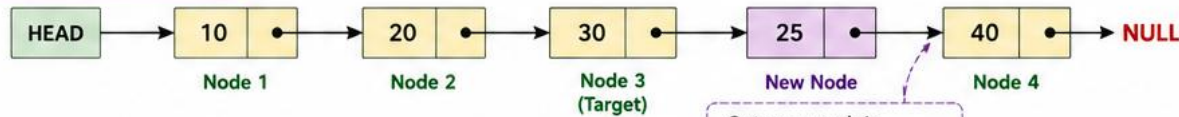
New node with
data = 25



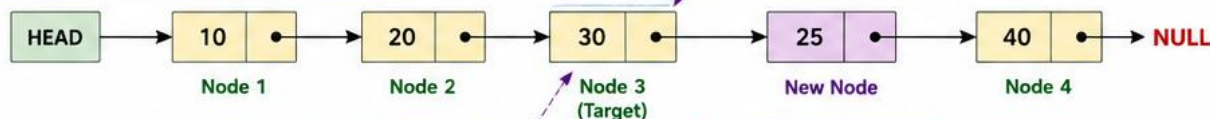
STEP 2: TRAVERSE TO FIND TARGET ELEMENT



STEP 3: LINK NEW NODE



STEP 4: UPDATE TARGET NODE'S NEXT



FINAL LINKED LIST

10 → 20 → 30 → 25 → 40 → NULL

ALGORITHM

- 1 Create a new node with the given data.
- 2 If the list is empty:
 - Set the new node as head and return.
- 3 Traverse the list to find the first occurrence of the given element.
- 4 If the element is not found, do not insert the new node.
- 5 Set new node's next to target node's next.
- 6 Set target node's next to the new node.
- 7 Insertion after the given element is complete.

TIME COMPLEXITY



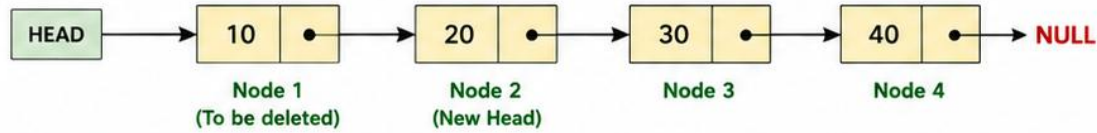
$O(n)$

In the worst case, we may need to traverse all n nodes to find the given element.

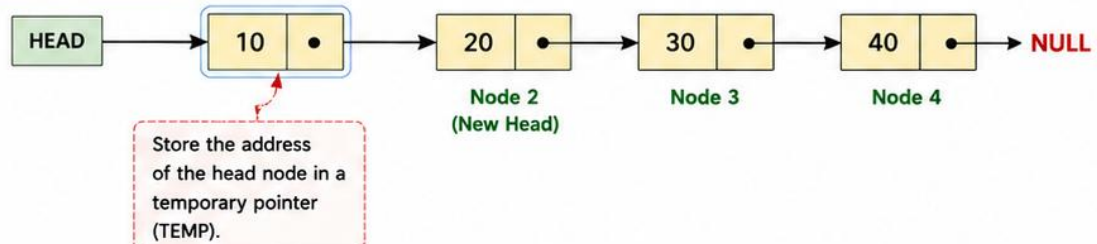
DELETE FROM START (BEGINNING) IN SINGLY LINKED LIST

Delete the first node (at the start) of the linked list and make the second node as the new head.

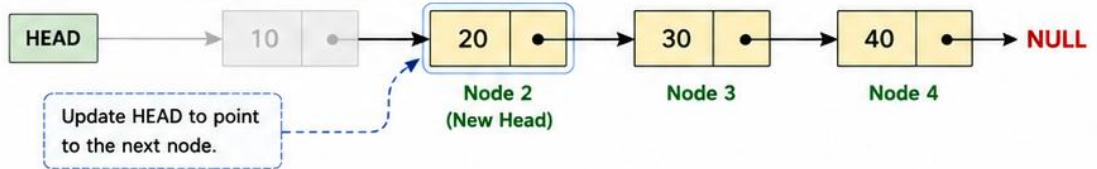
INITIAL LINKED LIST



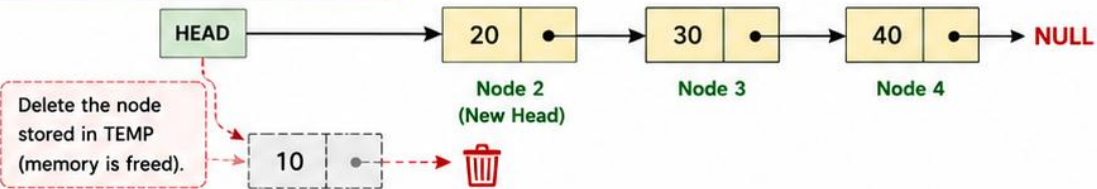
STEP 1: STORE THE HEAD NODE



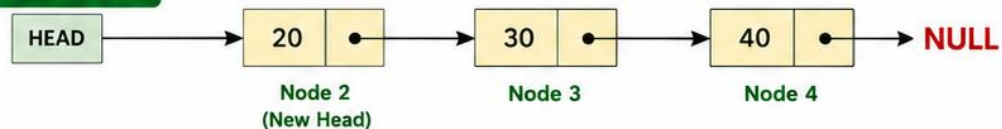
STEP 2: MOVE HEAD TO NEXT NODE



STEP 3: DELETE THE OLD HEAD NODE



FINAL LINKED LIST



ALGORITHM

- 1 If the list is empty (HEAD is NULL), do nothing.
- 2 Store the address of the head node in a temporary pointer (TEMP).
- 3 Move HEAD to the next node.
- 4 Delete the node stored in TEMP.
- 5 Deletion from start is complete.

TIME COMPLEXITY



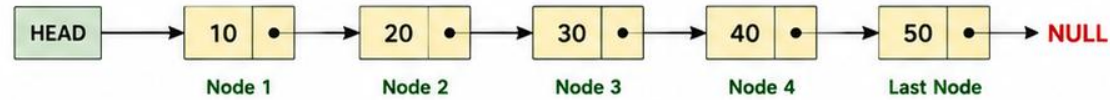
$O(1)$

We only perform a constant number of operations, regardless of the number of nodes in the list.

DELETE FROM END IN SINGLY LINKED LIST

Delete the last node (at the end) of the linked list.

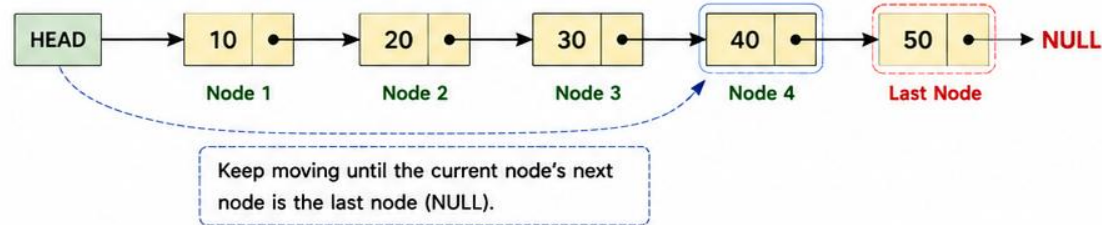
INITIAL LINKED LIST



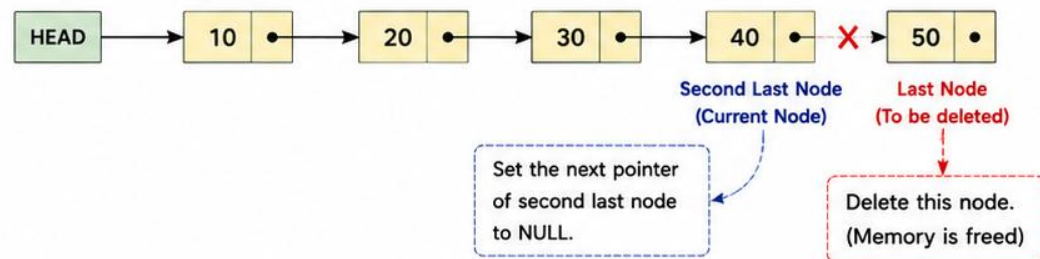
STEP 1: HANDLE EMPTY AND SINGLE NODE CASES

- If the list is empty (HEAD is NULL), do nothing.
- If there is only one node, delete it and set HEAD to NULL.

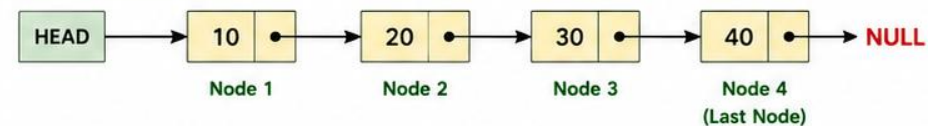
STEP 2: TRAVERSE TO FIND THE SECOND LAST NODE



STEP 3: DELETE THE LAST NODE



FINAL LINKED LIST



ALGORITHM

- 1 If the list is empty, do nothing.
- 2 If the list has only one node, delete it, set HEAD to NULL and return.
- 3 Start from HEAD and move to the next node until the next node is the last node (NULL). (Current node will be the second last node.)
- 4 Set the next of the current node (second last node) to NULL.
- 5 Delete the last node.
- 6 Deletion from end is complete.

TIME COMPLEXITY



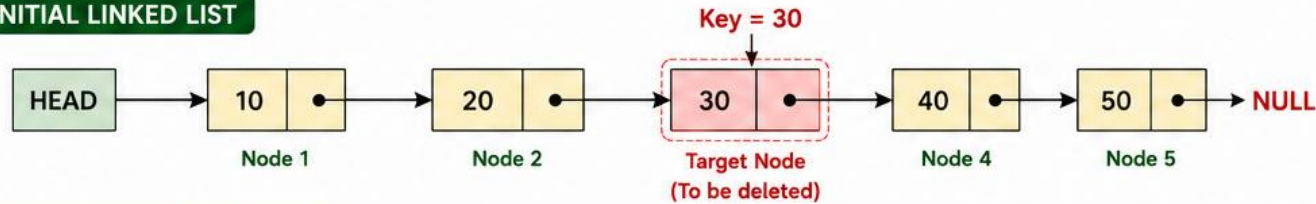
$O(n)$

We must traverse the list to reach the second last node. Therefore, the time complexity is $O(n)$.

DELETE A PARTICULAR NODE IN SINGLY LINKED LIST

Delete the first occurrence of the given key (value) from the linked list.

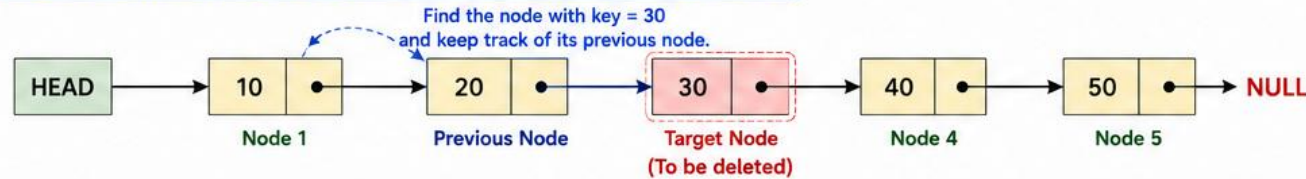
INITIAL LINKED LIST



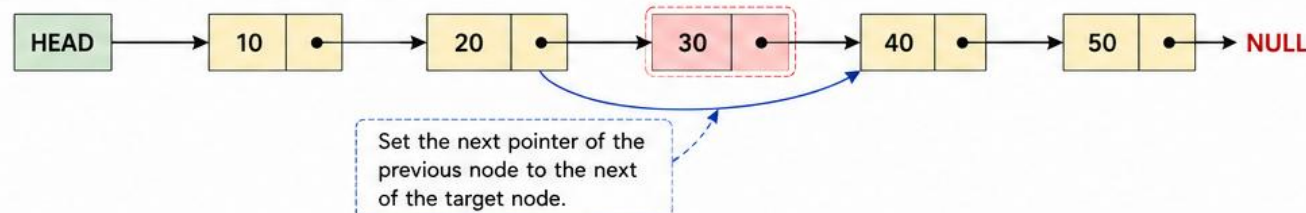
STEP 1: HANDLE EMPTY LIST

- If the list is empty (HEAD is NULL), do nothing.

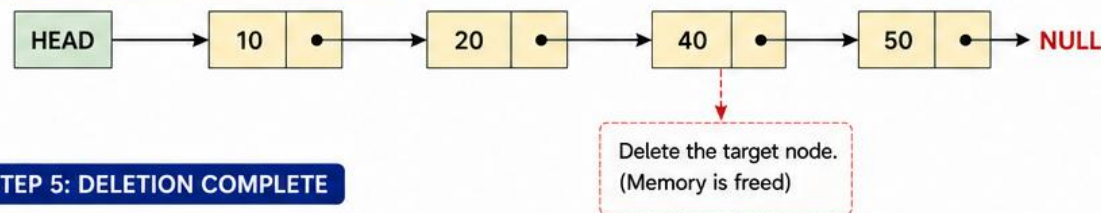
STEP 2: LOCATE THE NODE TO BE DELETED AND ITS PREVIOUS NODE



STEP 3: BYPASS THE TARGET NODE



STEP 4: DELETE THE TARGET NODE



STEP 5: DELETION COMPLETE

ALGORITHM

- 1 If the list is empty, do nothing.
- 2 If the head node contains the key:
 - Move HEAD to HEAD's next node.
 - Delete the old head node and return.
- 3 Otherwise, start from HEAD and traverse the list to find the node with the given key (target node) and its previous node.
- 4 If the target node is not found, do nothing.
- 5 Set the next pointer of the previous node to the next of the target node.
- 6 Delete the target node.
- 7 Deletion is complete.

TIME COMPLEXITY



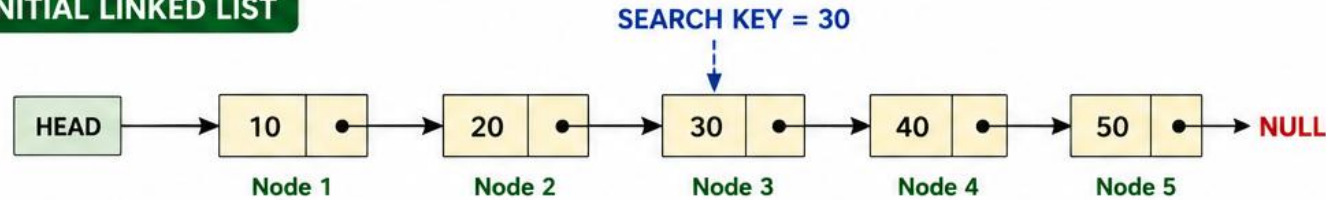
$O(n)$

In the worst case, we may need to traverse all n nodes to find the node with the given key.

SEARCH A NODE IN SINGLY LINKED LIST

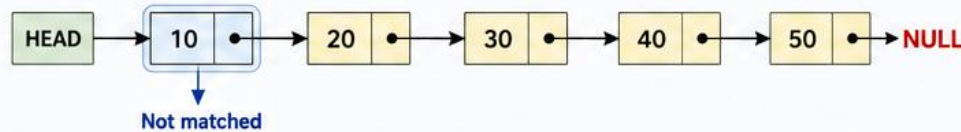
Search operation is used to find a node with a given key (value) in the linked list.

INITIAL LINKED LIST

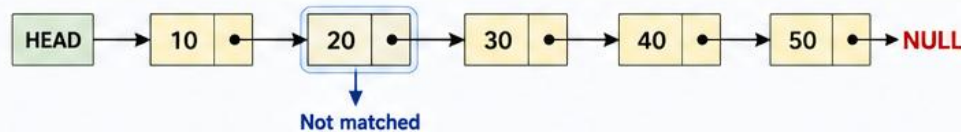


TRAVERSAL PROCESS

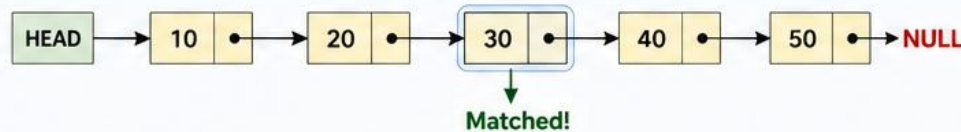
STEP 1 Start from head.
Compare 10 with 30.



STEP 2 Move to next node.
Compare 20 with 30.

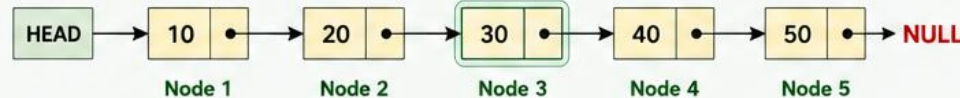


STEP 3 Move to next node.
Compare 30 with 30.



RESULT

Key 30 found at Node 3.
(Position / Index = 3)



NOT FOUND CASE

If the key is not present in the list,
the traversal will reach NULL.



ALGORITHM

- 1 Start from the head node.
- 2 Compare the data of the current node with the search key.
- 3 If the data matches the key:
 - Search is successful. (Return the node / position)
- 4 If not matched:
 - Move to the next node.
- 5 Repeat Steps 2 to 4 until either:
 - The key is found, or
 - The current node becomes NULL.
- 6 If the key is found, return success (and position of the node).
- 7 If the key is not found, return failure.
- 8 Stop.

TIME COMPLEXITY



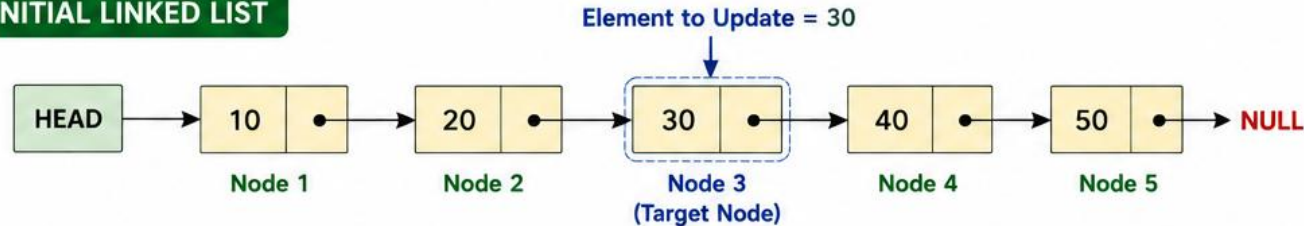
$O(n)$

In the worst case, we may need to visit
all n nodes to find the key or reach NULL.

UPDATE A NODE IN SINGLY LINKED LIST

Update means changing the data (value) of a node that contains the given element with a new value.

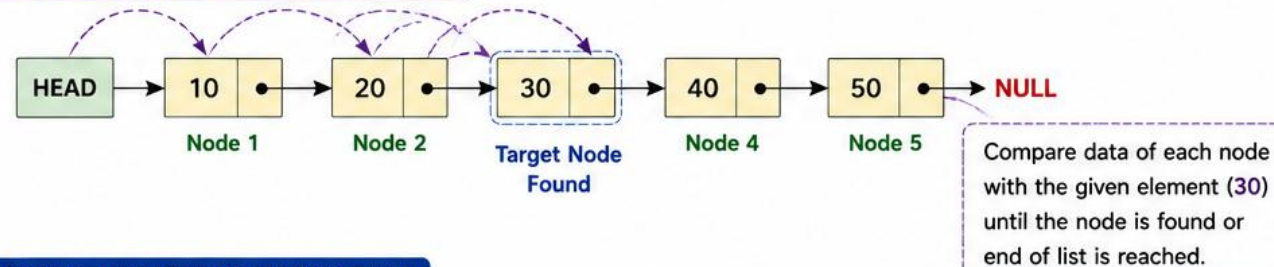
INITIAL LINKED LIST



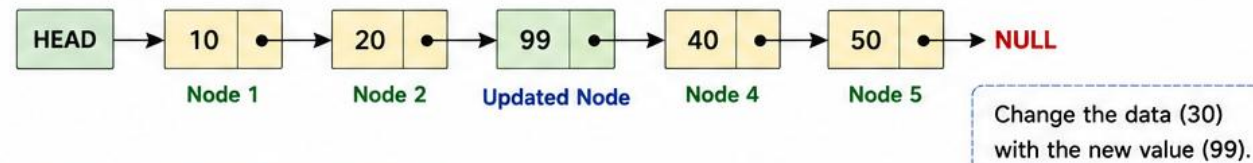
OPERATION

Update the node containing 30 with new value 99.

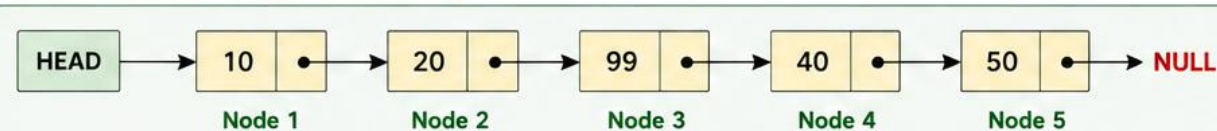
STEP 1: TRAVERSE TO FIND THE NODE



STEP 2: UPDATE THE NODE'S DATA



STEP 3: LINKED LIST AFTER UPDATE



ALGORITHM

- 1 Start from the head node.
- 2 Create a pointer (temp) and set it to head.
- 3 Traverse the list node by node.
- 4 Compare the data of current node with the given element.
- 5 If the element is found:
 - Update the node's data with the new value.
 - Stop the process.
- 6 If the element is not found and temp becomes NULL:
 - The element does not exist in the list.
- 7 Update operation is complete.

TIME COMPLEXITY



$O(n)$

In the worst case, we may need to traverse all n nodes to find the given element. Hence, the time complexity is $O(n)$.

Advantages of Linked List

- Dynamic size
- Efficient insertion at start
- Efficient deletion at start
- Better memory utilization
- Easy expansion

Prepared BY: Ehtisham Rasheed

Disadvantages of Linked List

- Extra memory for pointer
- Sequential access
- Slower searching
- Complex implementation
- No random access

Prepared BY: Ehtisham Rasheed

Which One Should We Choose?

- Suppose we're designing a software
- When should we choose Arrays?
- When should we choose Linked Lists?

Use Arrays When

- Frequent indexing
- Random access required
- Fixed number of elements
- Fast lookup required
- Examples
 - Student marks
 - Monthly sales
 - Matrix
 - Game board

Use Linked List When

- Frequent insertion
- Frequent deletion
- Unknown size
- Dynamic memory required
- Examples
 - Music playlist
 - Browser history
 - Undo feature
 - Polynomial representation

Practical Applications of Linked Lists

➤ Browser history

- Page 1 → Page 2 → Page 3
- Moving back, forward using links

➤ Music playlist

- Song 1 → Song 2 → Song 3
- Insert, delete songs any time

➤ Undo/Redo

- Word, photoshop, visual studio
- Every action stored as a node

Practical Applications of Linked Lists

➤ Image editor

- Layers
- can be inserted or removed dynamically

➤ Operating system

- Memory management, free blocks

➤ Graph representation

- Each vertex maintains a linked list

➤ Stack & Queue

- Can be implemented using linked list